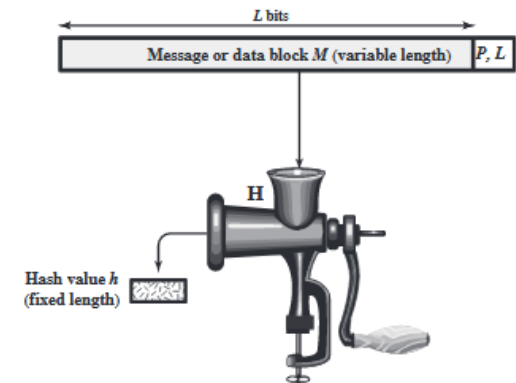


Module III
Chapter 1:
HASH FUNCTIONS
AND
MESSAGE AUTHENTICATION
CODES

Cryptographic Hash Functions

- A **cryptographic hash function** is a deterministic algorithm that takes an input of arbitrary length and produces a fixed-length output called a **hash value** or **message digest**.
- **Common Hash Functions**
 - MD5 (128-bit) – Broken
 - SHA-1 (160-bit) – Collision attacks exist
 - SHA-256 – Secure and widely used
 - SHA-3 – Latest NIST standard



P, L = padding plus length field

Figure 11.1 Cryptographic Hash Function; $h = H(M)$

Properties of Cryptographic Hash Functions

A secure hash function must satisfy:

1. Pre-image Resistance

- Given hash value h , it is computationally infeasible to find m such that: $H(m)=h$

2. Second Pre-image Resistance

- Given m_1 , it is infeasible to find $m_2 \neq m_1$ such that: $H(m_1)=H(m_2)$

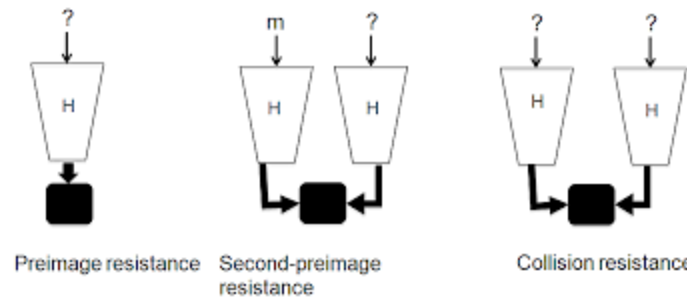
3. Collision Resistance

- It is infeasible to find **any two distinct messages** m_1 and m_2 such that: $H(m_1)=H(m_2)$

4. Avalanche Effect

- Small change in input $\rightarrow$ drastic change in output.

5. Deterministic & Efficient



Birthday Attack

- The **birthday attack** is a cryptographic attack based on probability theory — specifically the **Birthday Paradox**.
- It mainly targets **hash functions** like:
 - MD5
 - SHA-1
 - SHA-256
- It is used to find **collisions** in hash functions.

Based on the **Birthday Paradox**.

- If hash output length = **n bits**, then collision complexity $\approx 2^{n/2}$

Example:

- 128-bit hash $\rightarrow$ collision in about 2^{64} attempts.
- In a group of just **23 people**, there is more than **50% chance** that two people share the same birthday. Even though:
 - There are 365 possible birthdays.
 - 23 is much smaller than 365.

Example:

Suppose a hash function produces **3-digit hash values**

Messages:

M1 = "Hello"

M2 = "Good"

M3 = "Cat"

M4 = "Dog"

M5 = "Apple"

Hash Values:

$H(\text{"Hello"}) = 321$

$H(\text{"Good"}) = 874$

$H(\text{"Cat"}) = 321$

$H(\text{"Dog"}) = 542$

$H(\text{"Apple"}) = 900$

Collision Found:

- $H(\text{"Hello"}) = 321$
 $H(\text{"Cat"}) = 321$
- Two **different messages produce the same hash value.**
- This is called a **collision**, and the attacker has successfully performed a **Birthday Attack**.

Practical Example in Cryptography

- Suppose a digital signature system signs the hash of a message.

Step 1

- Attacker creates **two message sets**
- Set A (legitimate looking)
Set B (fraudulent)

Step 2

- Attacker keeps generating messages until:
- $\text{Hash}(\text{Message-A}) = \text{Hash}(\text{Message-B})$

Step 3

- Victim signs **Message-A**.

Step 4

- Attacker replaces it with **Message-B** because both hashes are the same.
- $\text{Sign}(\text{Hash}(\text{Message-A})) = \text{Sign}(\text{Hash}(\text{Message-B}))$
- Thus the **fraudulent message becomes valid**.

How Birthday Attack Works:

Let's say attacker wants a collision.

Step 1:

- Generate random input A_1
Compute hash $H(A_1)$

Step 2:

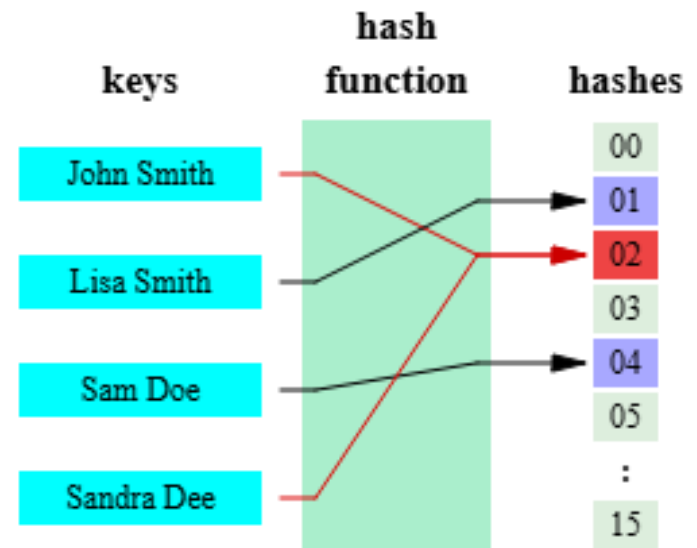
- Generate random input A_2
Compute hash $H(A_2)$

Step 3:

- Store all hashes in a table.

Step 4:

- Keep generating inputs until:
- $H(A_i) = H(A_j)$
where $i \neq j$
- When match occurs $\rightarrow$ Collision found.



Why This is Dangerous?

Because collision can allow:

- Forging digital signatures
- Fake certificates
- Tampered documents
- Breaking blockchain integrity (if weak hash used)
- Example:
A digital certificate signed with weak hash (like SHA-1) can be manipulated.

Types of Birthday Attacks

- Classic Collision Attack
- Chosen-Prefix Collision Attack
- Multi-Collision Attack

Advanced attacks exploit structure weaknesses in specific algorithms.

Why SHA-256 is Safe (Currently)

For SHA-256:

- Output size = 256 bits
 - Birthday complexity = 2^{128}
 - 2^{128} is astronomically large.
 - So practically infeasible today.
-
- The Birthday Attack demonstrates that the effective security level of a hash function against collisions is only **half its output length**.
 - Therefore, cryptographic systems must use sufficiently large hash sizes to ensure practical collision resistance.
 - The attack highlights the importance of strong hash functions in maintaining integrity and authentication in modern cryptographic applications.

Hashcash

- **Hashcash** is a cryptographic **proof-of-work (PoW)** system proposed by Adam Back in 1997.
- It was originally designed to prevent **email spam** and **denial-of-service (DoS)** attacks by making the sender perform a small amount of computational work before sending a message.
- Later, the same concept became the foundation of Bitcoin and other blockchain systems.
- Hashcash requires the sender to compute a hash value with a specific property:
 - The hash output must begin with a certain number of leading zeros.

Working Mechanism

Step 1: Create a Stamp

- The sender constructs a string containing:
 - Version number
 - Date
 - Resource (e.g., email address)
 - Random nonce
 - Counter value
- Example format: ver:bits:date:resource:random:counter

Step 2: Compute Hash

- The sender repeatedly changes the **counter** and computes the hash:
 $H(\text{stamp}) = \text{hash value}$
- Until the hash output satisfies: **Hash begins with k leading zeros**

Step 3: Send Message

- The valid stamp is attached to the email/message.

Step 4: Verification

- The receiver:
 - Computes hash once
 - Checks if required number of leading zeros exist
- Verification is fast ($O(1)$), but generation requires approximately:
 2^k hash attempts

Mathematical Perspective

- If the difficulty level requires **k leading zero bits**, then:
- Expected attempts= 2^k
- Example:
 - $k = 20$
 - Expected computations $\approx 2^{20} \approx 1$ million attempts
 - This is manageable for a single user but expensive for spammers sending millions of emails.

Applications

- 1. Spam Prevention:** Makes bulk email sending computationally expensive.
- 2. DoS Attack Mitigation:** Prevents attackers from flooding servers with requests.
- 3. Blockchain Systems:** Hashcash inspired Bitcoin's Proof-of-Work mechanism.

In Bitcoin:

- Miners must find a hash below a target value.
- This is essentially a large-scale Hashcash system.

Advantages	Limitations
• Simple concept • Easy verification • Adjustable difficulty • Reduces automated abuse	• Consumes computational power • Energy inefficient at large scale • Not suitable for low-power devices • Can disadvantage users with slower hardware

Difference Between Hashcash and Birthday Attack

Feature	Hashcash	Birthday Attack
Purpose	Prevent spam	Find hash collisions
Concept	Proof-of-Work	Probability theory
Complexity	(2^k)	$(2^{\{n/2\}})$
Nature	Defensive	Offensive

SECURE HASH ALGORITHM (SHA)

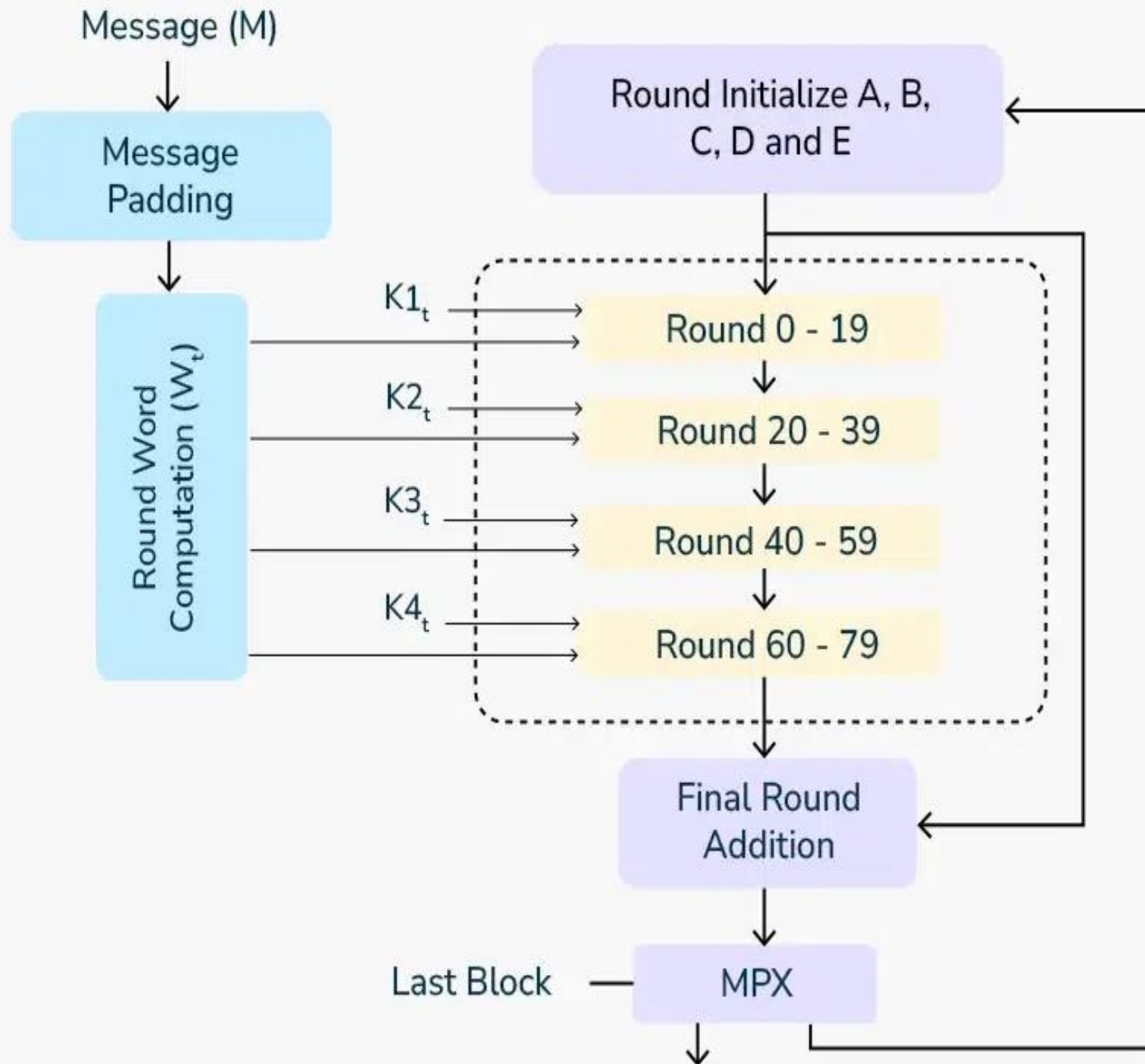
- A Secure Hash Algorithm (SHA) is a cryptography technique and are developed by **National Security Agency(NSA)** and at a later stage, the **National Institute of Standards and Technology** standardized the SHA technique and ready for various purposes(authentication, message integrity checks, digital signatures, and key derivation)
- SHA is a **modified version of MD5** and used for **hashing data and certificates**.
- A hashing algorithm shortens the input data into a smaller form that cannot be understood by **using bitwise operations, modular additions, and compression functions**.
- A few algorithms of interest are SHA-1, SHA-2, and SHA-3, each of which was successively designed with increasingly stronger encryption in response to hacker attacks.
- SHA-0, for instance, is now obsolete due to the widely exposed vulnerabilities.

SHA-1 Hash

- SHA-1 or Secure Hash Algorithm 1 is a cryptographic algorithm that **takes an input and produces a 160-bit (20-byte) hash value.**
- This **hash value is known as a message digest.**
- This **message digest** is usually then rendered as a **hexadecimal number which is 40 digits long.**
- SHA-1 is been considered insecure since 2005. Major tech giants browsers like Microsoft, Google, Apple, and Mozilla have stopped accepting SHA-1 SSL certificates by 2017.

Applications:

- Cryptography
- Data Integrity
- Digital Signatures
- Digital Forensics
- Password Storage
- Software Updates



Components and Process Flow:

1. Message (M):

- The original input message that needs to be hashed.

2. Message Padding:

- The initial step where the message is padded to ensure its length is congruent to 448 modulo 512. This step prepares the message for processing in 512-bit blocks.

3. Round Word Computation (Wt):

- After padding, the message is divided into blocks of 512 bits, and each block is further divided into 16 words of 32 bits. These words are then expanded into 80 32-bit words, which are used in the subsequent rounds.

4. Round Initialize (A, B, C, D, and E):

- Initialization of **five working variables** (A, B, C, D, and E) with specific constant values. These variables are used to compute the hash value iteratively.

5. Round Constants (Kt):

- SHA-1 uses four constant values (K1, K2, K3, K4), each applied in a specific range of rounds:
 - K1 for rounds 0-19
 - K2 for rounds 20-39
 - K3 for rounds 40-59
 - K4 for rounds 60-79

6. Rounds (0-79):

- The main computation loop of SHA-1, divided into four stages (each corresponding to one of the constants K1 to K4). In each round, a combination of logical functions and operations is performed on the working variables (A, B, C, D, and E) using the words generated in the previous step.

7. Final Round Addition:

- After all 80 rounds, the resulting values of A, B, C, D, and E are added to the original hash values to produce the final hash.

8. MPX (Multiplexing):

- Combines the results from the final round addition to form the final message digest.

Summary of Steps:

- **Input (Message M):** The process starts with the input message.
- **Message Padding:** The message is padded to meet the length requirements.
- **Word Computation:** The padded message is divided into blocks and further into words, which are expanded for use in the rounds.
- **Initialization:** Initial hash values are set.
- **Round Processing:** The main loop performs 80 rounds of computation using the message words and round constants.
- **Final Addition:** The results from the rounds are added to the initial hash values.
- **Output (Hash Value):** The final message digest is produced.

MESSAGE AUTHENTICATION CODES (MAC)

- A **Message Authentication Code (MAC)** is a cryptographic technique used to ensure **data integrity and message authentication**.
- MACs use a **secret key** combined with the message to produce a short authentication value called a **tag** or **MAC value**.
- A MAC function operates as follows: **$MAC=f(K,M)$**
- Where:
 - **K** → Secret key shared between sender and receiver
 - **M** → Message
 - **MAC** → Authentication tag

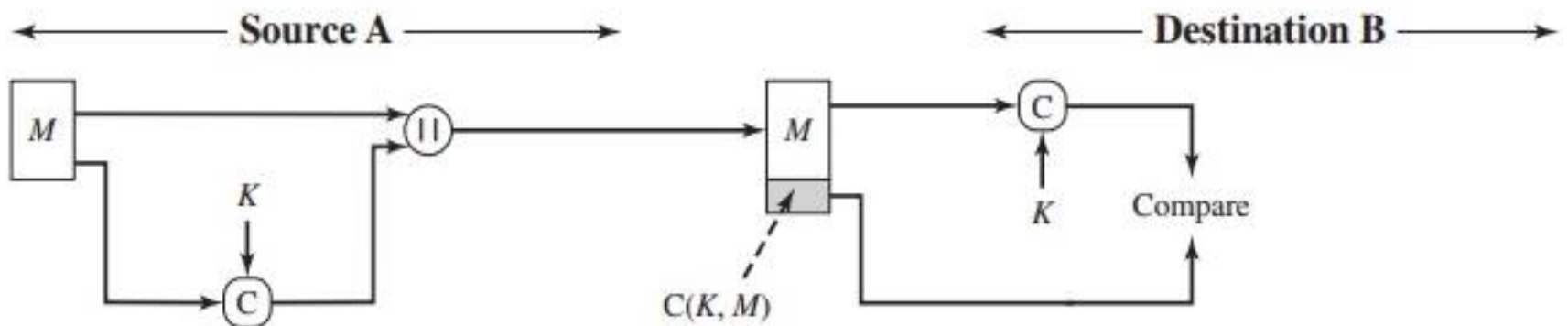
Process:

- **Sender side**
 - Sender computes MAC using secret key and message.
 - Sender sends **message + MAC**.
- **Receiver side**
 - Receiver re-computes MAC using same key.
 - If both MAC values match → message is authentic.

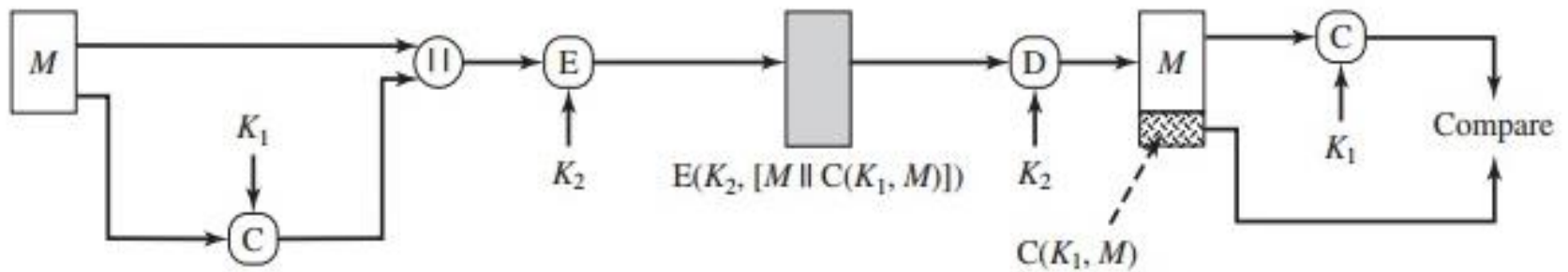
Properties of MAC

A secure MAC should provide:

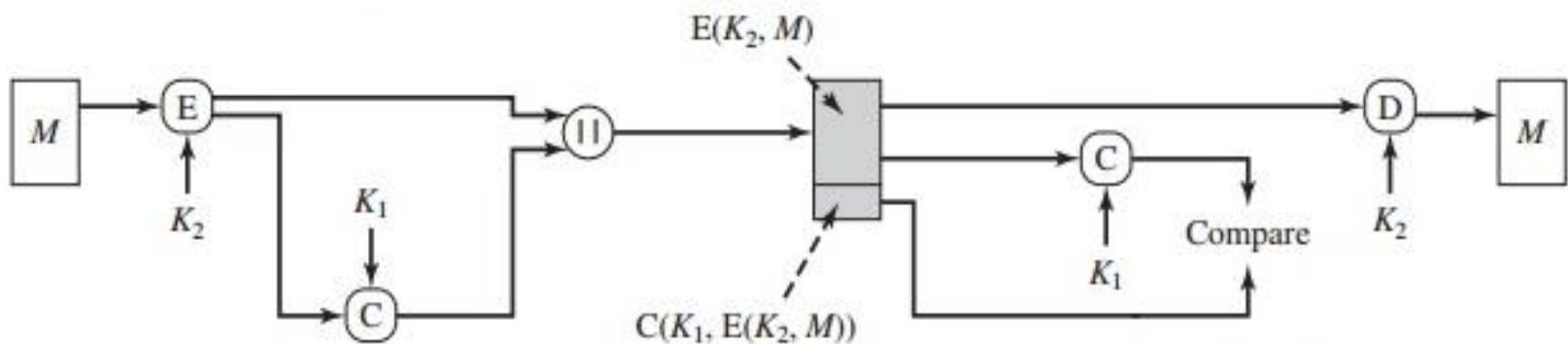
- **Data Integrity** – Detects modification of message
- **Authentication** – Confirms sender identity
- **Key-based security** – Only parties with secret key can generate valid MAC
- **Resistance to forgery**



(a) Message authentication



(b) Message authentication and confidentiality; authentication tied to plaintext



(c) Message authentication and confidentiality; authentication tied to ciphertext

Figure 12.4 Basic Uses of Message Authentication code (MAC)

MAC Protocols

- Two important MAC protocols are:
 - **HMAC (Hash-based MAC)**
 - **CMAC (Cipher-based MAC)**

HMAC Algorithm

- **HMAC algorithm** stands for **Hashed or Hash-based Message Authentication Code**.
- It is a result of work done on **developing a MAC derived from cryptographic hash functions**.
- HMAC is a **great resistance to cryptanalysis attacks** as it uses the **Hashing concept twice**.
- HMAC consists of **twin benefits of Hashing and MAC** and thus is **more secure than any other authentication code**.
- **RFC 2104** has issued HMAC, and HMAC has been made compulsory to implement in IP security.
 - (**Request for Comments**: It is a type of official document published by the Internet Engineering Task Force that describes standards, protocols, and guidelines used on the Internet.)
- The **FIPS 198** NIST standard has also been issued by HMAC.
 - (**Federal Information Processing Standards**: It is a set of official standards developed by the National Institute of Standards and Technology (NIST), which is part of the U.S. government.)

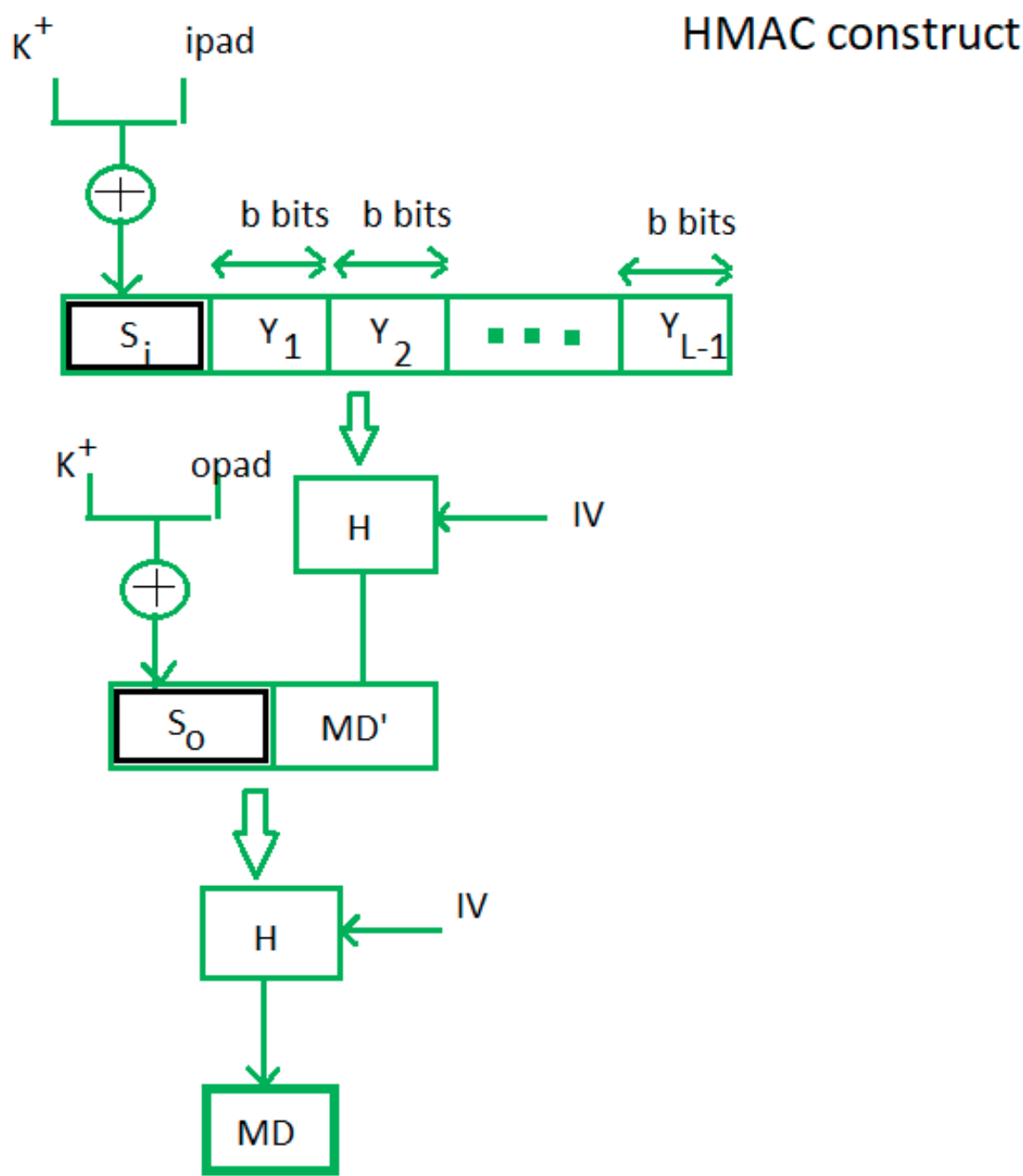
Objectives of HMAC

- **One way**, i.e., easy to generate output from input but complex the other way around.
- **Less affected by collisions** than the hash functions.
- HMAC **reuses algorithms like MD5 and SHA-1** and checks to replace the embedded hash functions with more secure hash functions, in case found.
- HMAC tries to **handle the Keys in a more simple manner**.

Working of HMAC Algorithm

- Secret key is padded to block size.
- Key is XORed with **ipad** and combined with message.
- Inner hash is computed.
- Result is XORed with **opad** and hashed again.
- Final output is the **MAC tag**.

Here is a simple structure of HMAC:



- Here, H stands for Hashing function,
- M is the original message
- S_i and S_o are input and output signatures respectively,
- Y_i is the i th block in original message M, where i ranges from $[1, L)$
- L = the count of blocks in M
- K is the secret key used for hashing
- IV is an initial vector (some constant)
- The generation of input signature and output signature S_i and S_o respectively.

$$S_i = K^+ \oplus \text{ipad}$$

where K^+ is nothing but K padded with zeros on the left so that the result is b bits in length

$$S_o = K^+ \oplus \text{opad}$$

where ipad and opad are 00110110 and 01011100 respectively taken $b/8$ times repeatedly.

$$MD' = H(S_i || M)$$

$$MD = H(S_o || MD') \quad \text{or} \quad MD = H(S_o || H(S_i || M))$$

Advantages of HMAC

- Strong security
- Resistant to length extension attacks
- Easy to implement
- Works with existing hash functions

Disadvantages of HMAC

- HMAC uses a **shared secret key**, which can cause non-repudiation issues. If the **secret key is compromised**, attackers can generate fake authenticated messages.
- **Secure key distribution and management** between parties is difficult.
- **Hash collisions**, though rare, can still occur in hash functions.
- **Short secret keys** are vulnerable to brute-force attacks.
- The security of HMAC depends on the **strength of the underlying hash function** (e.g., SHA-256).

Applications of HMAC

- **Verification of e-mail address** during activation or creation of an account.
- **Authentication of form data** that is sent to the client browser and then submitted back.
- HMACs can be used for **Internet of Things (IoT) due to less cost**.
- Whenever there is a need to reset the password, a link that can be used once is sent without adding a server state.
- It can take a message of any length and convert it into a fixed-length message digest. That is even if you got a long message, the message digest will be small and thus permits maximizing bandwidth.

CMAC (Cipher-Based Message Authentication Code)

- CMAC is a MAC algorithm based on **block cipher encryption** rather than hash functions.
- It is based on the block cipher:
 - AES
- CMAC is an improved version of **CBC-MAC**.

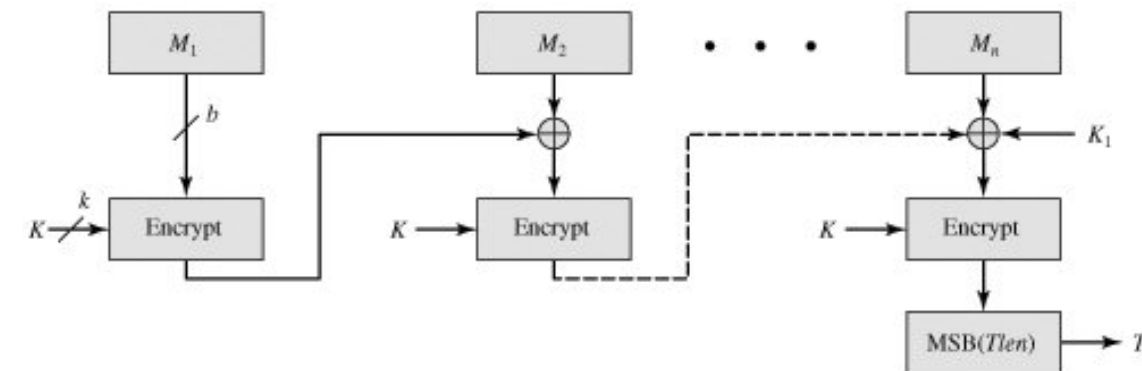
Working of CMAC:

- Message is divided into fixed-size blocks.
- Blocks are processed using block cipher encryption.
- Special **subkeys (K1 and K2)** are generated from the main key.
- Final block is XORed with subkey before encryption.
- The output of last encryption becomes the **MAC tag**.

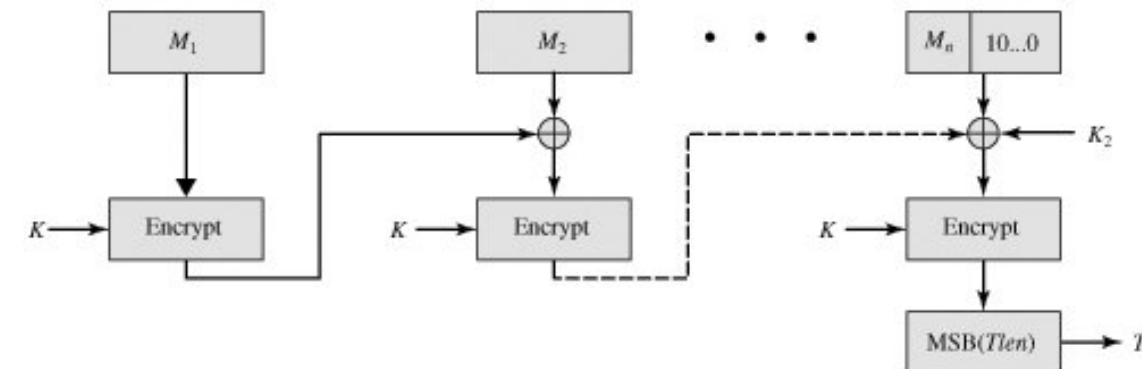
CMAC Process

Steps:

- Generate sub-keys **K1** and **K2**.
- Divide message into blocks.
- Apply XOR with previous block result.
- Encrypt using block cipher.
- Final encryption output = MAC.



(a) Message length is integer multiple of block size



(b) Message length is not integer multiple of block size

□ **CMAC** is calculated as follows:

$$C_1 = E(K, M_1)$$

$$C_2 = E(K, [M_2 \oplus C_1])$$

$$C_3 = E(K, [M_3 \oplus C_2])$$

⋮

⋮

⋮

$$C_n = E(K, [M_n \oplus C_{n-1} \oplus K_1])$$

$$T = \text{MSB}_{Tlen}(C_n)$$

T = message authentication code, also referred to as the tag

$Tlen$ = bit length of T

$\text{MSB}_s(X)$ = the s leftmost bits of the bit string X

Features of CMAC:

- Strong Security
- Fixed-Length Output
- Efficient
- Key Reuse
- Easy Implementation

Advantages of CMAC:

- High security
- Based on well-tested block ciphers
- Standardized by NIST
- Efficient for hardware implementations

Limitations of CMAC:

- Limited Key Size
- Vulnerability to Side-Channel Attacks
- Limited Block Size
- Key Management

Applications of CMAC:

- **Secure Communication:** CMAC is used to authenticate messages in secure communication protocols such as SSL/TLS, IPSec, and SSH.
- **File Integrity:** CMAC is used to verify the integrity of files and ensure that they have not been tampered with.
- **Digital Signatures:** CMAC is used to generate digital signatures, which are used to verify the authenticity of documents and messages.
- **Payment Systems:** CMAC is used in payment systems such as credit cards and electronic wallets to authenticate transactions and prevent fraud.

Comparison of HMAC and CMAC

Feature	HMAC	CMAC
Based on	Hash function	Block cipher
Example algorithm	SHA-256	AES
Security basis	Hash security	Cipher security
Speed	Faster in software	Efficient in hardware
Usage	Internet protocols	Embedded systems

MACs based on block ciphers: DAA and CMAC:

There are two MACs that are based on the use of a block cipher mode of operation: **The Data Authentication Algorithm (DAA)**, which is now obsolete, and the **Cipher-Based Message Authentication Code (CMAC)**, which is designed to overcome the deficiencies of the DAA.

Data Authentication Algorithm

- The Data Authentication Algorithm (DAA), **based on DES**, has been one of the most widely used MACs for a number of years.
- The algorithm is both a **FIPS publication (FIPS PUB 113)** and an **ANSI standard (X9.17)**.
- The algorithm can be defined as using the **cipher block chaining (CBC)** mode of operation of DES with an initialization vector of zero.
- The data (e.g., message, record, file, or program) to be authenticated are grouped into contiguous 64-bit blocks: $D_1, D_2, \dots, D_N$.
- If necessary, the final block is padded on the right with zeroes to form a full 64-bit block.
- Using the DES encryption algorithm E and a secret key K , a data authentication code (DAC) is calculated as follows

